

SurfSplat: Conquering Feedforward 2D Gaussian Splatting with Surface Continuity Priors

ICLR 2026

Bing He¹, Jingnan Gao¹, Yunuo Chen¹, Gang Chen², Ning Cao²,

Zhengxue Cheng¹, Li Song¹, Wenjun Zhang¹

¹Shanghai Jiao Tong University ²Tianyi Shilian Technology Co., Ltd

2026.02.12

Contents

1. Introduction
2. Method
3. Experiments
4. Conclusion

Introduction

- NVS(Novel View Synthesis)의 방식에서도, Gaussian Splatting은 크게 2가지로 나눌 수 있음
 - COLMAP input을 받아 optimization을 통해 scene을 생성하는 방식
 - Feedforward 방식으로, 사전학습 모델을 사용하여 single/multi-view에서 바로 Gaussian을 생성하는 방식
- 다만, 기존의 feedforward 방식들은 빠르긴 하지만, rendering 품질이 떨어질 수 있음
 - 최적화를 통해 gaussian의 속성을 수정하고, scene의 품질을 끌어올렸지만, 이 부분이 사라짐
 - 또한, 3D Gaussian을 만드는 데, 2D image loss에 의존
 - 즉, image가 없는 곳(한번도 본 적이 없던 곳)은 gaussian이 부정확하게 생성될 수 있음
- 따라서, 본 논문에서는 surface continuity prior를 도입하여, feedforward Gaussian splatting의 품질을 끌어올림
- 또한, forced alpha blending을 통해, occlusion 문제도 해결
- 마지막으로 HRRC(high resolution redering-based metric)을 도입하여 평가

Method

Encoder

- Multi-view branch와 Single View branch로 나누어 구성 (Depth Splat과 유사)
- 각 이미지에 ResNet 기반 stride-2 layer 2개를 사용해 1/4 사이즈의 feature map을 추출
 - 이 feature들을 multi-view Swin Transformer에 통과시켜, self, cross attention 수행을 6번 반복한 결과는 아래와 같음

$$\{F^i\}_{i=1}^N, F^i \in \mathbb{R}^{\frac{H}{s} \times \frac{W}{s} \times C}$$

- 이 Multi-view aware output을 plane sweep stereo를 통해 $\{F_{d_m}^{j \rightarrow i}\}$ 마다 score를 뽑아서 평균을 냄(source image가 2개면 score/2, 3개면 score/3)
- 이후 Softmax 통과시켜 cost volume을 확률밀도값으로 가지고있음 $C_i \in \mathbb{R}^{\frac{H}{s} \times \frac{W}{s} \times D}$ 생성

Method

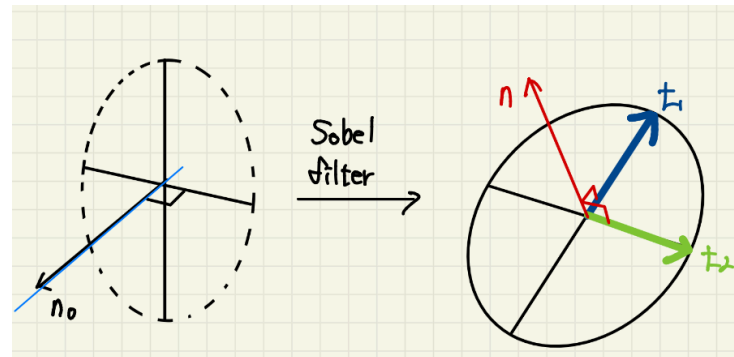
- Single-view branch는 DepthAnything V2를 사용하면서, texture가 없거나, correspondence가 부족한 영역에서 일단 gaussian을 생성
 - Depth Anything V2 는 1/14 사이즈의 결과가 나오는데, 1/4로 bilinear upsampling 후 사용
 - 최종 output은 monocular features $F_m^i \in \mathbb{R}^{\frac{H}{s} \times \frac{W}{s} \times C_m}$
- 두 cost volume을 concat해 U-Net에 입력하여 Gaussian attributes 추출
 - texture가 없는곳은 Single view branch의 이점을, depth가 모호한 곳은 multi-view branch의 이점을 취함

$$f_{\theta} : \{(I^v, \mathbf{k}^v, \mathbf{T}^v)\}_{v=1}^V \mapsto \left\{ \bigcup_{j=1}^{H \times W} (\boldsymbol{\mu}_j^v, \alpha_j^v, \mathbf{r}_j^v, \mathbf{s}_j^v, \mathbf{c}_j^v) \right\}_{v=1}^V$$

- 여기서 초기 3D Gaussian을 만들어냄!

Method

Surface Continuity Prior



- 기존의 feedforward 방식들은 2D image loss에만 의존하여, 3D Gaussian을 생성
 - 이미지가 없는 곳에서는 부정확한 Gaussian이 생성될 수 있음; 이것 **scene이 불연속적이다** 라는 표현을 쓰기도 함
- 본 논문에서는, 대부분의 실제 표면은 locally smooth하고, 이미지에서 이웃한 픽셀은 3D 표면에서도 이웃한 점일 가능성이 높다는 것을 가정
 - 즉, 이미지 상에서 이웃한 pixel들은, 3D 공간에서도 이웃한 Gaussian을 만들어야 한다는 가정
 - 즉, rotation, scale은 이웃한 Gaussian과 유사해야 함
- 3차원 공간에서의 Gaussian은 $\mathbf{p}_0 \in \mathbb{R}^3$, normal vector는 COLMAP의 좌표계 기준에 따라 $\mathbf{n}_0 = (0, 0, 1)^\top$, rotation은 $\mathbf{R}_0 \in SO(3)$
 - 즉, Gaussian은 카메라 view direction과 정면에 위치하도록 설정
- 정밀한 local surface normal을 얻기 위해, depth map에 Sobel filter를 수행(x, y 축으로 sobel filtering) 후 virtual neighbors 생성($\mathbf{p}_1, \mathbf{p}_2$)
 - 기준점으로부터 빼서 plane의 기저벡터 생성

$$\mathbf{t}_1 = \mathbf{p}_1 - \mathbf{p}_0, \quad \mathbf{t}_2 = \mathbf{p}_2 - \mathbf{p}_0$$

Method

- 이후, 두 벡터의 외적을 통해 normal vector 계산

$$\mathbf{n} = \frac{\mathbf{t}_1 \times \mathbf{t}_2}{\|\mathbf{t}_1 \times \mathbf{t}_2\|}$$

- 계산된 normal을 기준으로, 기존의 $\mathbf{n}_0$ 를 $\mathbf{n}$ 으로 회전시키는 rotation matrix $\mathbf{R}$ 계산

$$\mathbf{R} = \mathbf{I} + [\mathbf{v}]_{\times} + \frac{1 - c}{\|\mathbf{v}\|^2} [\mathbf{v}]_{\times}^2$$

$$\mathbf{R}_{\text{surf}} = \mathbf{R}\mathbf{R}_0 = \mathbf{R}$$

- Scale은 앞서 구한 plane에 주변 point들을 투영한 후 각 축으로의 분산 계산

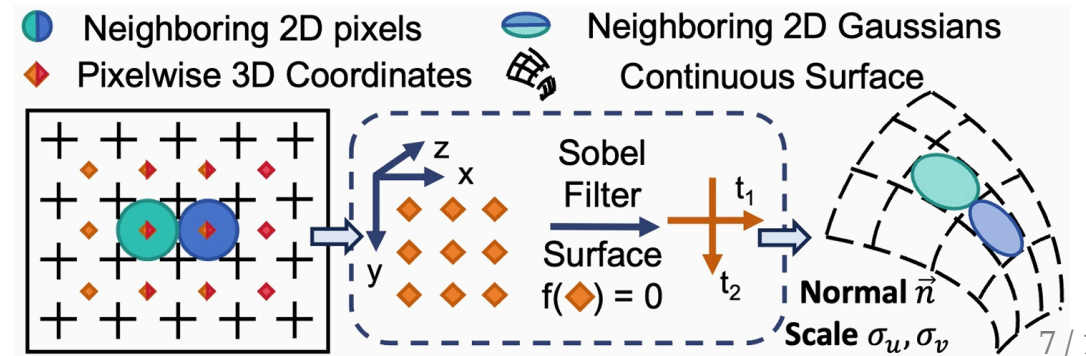
$$u = \frac{t_1}{\|t_1\|}, \quad v = \frac{t_2}{\|t_2\|}, \quad w = n \text{ (Normal)}$$

$$\text{dist}_{u,k} = (p_k - p_0) \cdot u$$

$$\text{Var}_u = \frac{1}{N} \sum_{k \in \mathcal{N}} ((p_k - p_0) \cdot u)^2$$

$$\sigma_u = \sqrt{\text{Var}_u} = \sqrt{\frac{1}{N} \sum_{k \in \mathcal{N}} ((p_k - p_0) \cdot u)^2}$$

- 2D Gaussian이기에 $S = \text{diag}(\sigma_u, \sigma_v, \sigma_w = 0)$



Method

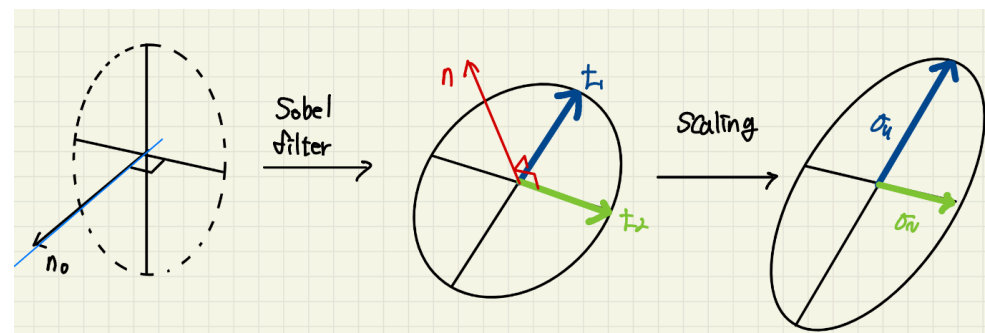
- Covariance는 앞서 구한 Rotation과 Scale을 사용하여 다음과 같이 계산

$$\Sigma = \mathbf{R} \mathbf{S} \mathbf{S}^T \mathbf{R}^T,$$

$$\Sigma' = \mathbf{J} \mathbf{W} \Sigma \mathbf{W}^T \mathbf{J}^T,$$

- But,, 앞에 설명한 '정석' 대로 Projection Jacobian을 역으로 풀어 3D scale을 구하는 방식은 불안정할 수 있다.
 - Feedforward 방식에서는 initial point도 없고, 매 pixel마다 독립적으로 Gaussian을 생성해야 함
 - 따라서, tangent plane 상에서 이미 관측된 vector들의 크기를 이용해 2D Gaussian의 scale을 이용해 근사(coarse scale estimation)
 - 기존에 구한 t_1, t_2 벡터의 각 성분을 활용해 scale 결정
 - 즉, surface-consistent scale을 먼저 만들고, multiplier로 fine-tuning 한다.

$$\begin{aligned} \bar{\sigma}_u^2 &= t_{1x}^2 + t_{1z}^2, & \bar{\sigma}_v^2 &= t_{2y}^2 + t_{2z}^2 \\ \sigma_u &= \bar{\sigma}_u \hat{\sigma}_u, & \sigma_v &= \bar{\sigma}_v \hat{\sigma}_v \end{aligned}$$



Method

Forced Alpha Blending

- 기존의 alpha blending은 일정 gaussian의 opacity가 커지면, 뒤의 gaussian이 가려져 해당 pixel에 아예 영향이 없는 경우가 생김
 - 기존 alpha blending

$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad \alpha = \sum_{i \in \mathcal{N}} \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j).$$

- 위 식대로라면, geometry가 있어서 gaussian이 생성되어야 하는 경우에 training view에서 보이지 않으면 gaussian이 생성되지 않음
- 따라서, alpha 값을 일정 값 이상 커지지 않도록 강제(force) 하도록 함 ($\alpha < \tau_{opa} < 1$)
- 또한, alpha값에 따라 color contribution도 조절
 - alpha 값이 큰 녀석은 복원해주도록 함 (e.g., $C/\alpha = (0.6, 0, 0)/0.6 = (1, 0, 0)$)

$$C = \begin{cases} C, & \alpha < \tau_{\alpha}, \\ \frac{C}{\alpha}, & \alpha \geq \tau_{\alpha}, \end{cases}$$

Method

Training loss

- 기존의 3DGS와 유사하게, image rendering 기반 loss

$$L_{gs} = \sum_{m=1}^M \left(\text{MSE} \left(I_{\text{render}}^m, I_{\text{gt}}^m \right) + \lambda \cdot \text{LPIPS} \left(I_{\text{render}}^m, I_{\text{gt}}^m \right) \right),$$

HRRC(High-Resolution Rendering Consistency)

- 새로운 metric을 제안하는데, 기존의 metric은 저해상도 이미지에서 계산
- 따라서, high-resolution rendering을 통해, 더 정밀한 평가를 수행
- 2배, 4배 upsample한 이미지에서 각각 PSNR, SSIM, LPIPS 계산($\times 1$, $\times 2$, $\times 4$)

$$\text{HRRC}_{\text{metric}} = \text{metric}(\hat{I}^{HR}, \hat{I}^{GT\uparrow}) \quad \text{where metric} \in \{\text{PSNR}, \text{SSIM}, \text{LPIPS}\}$$

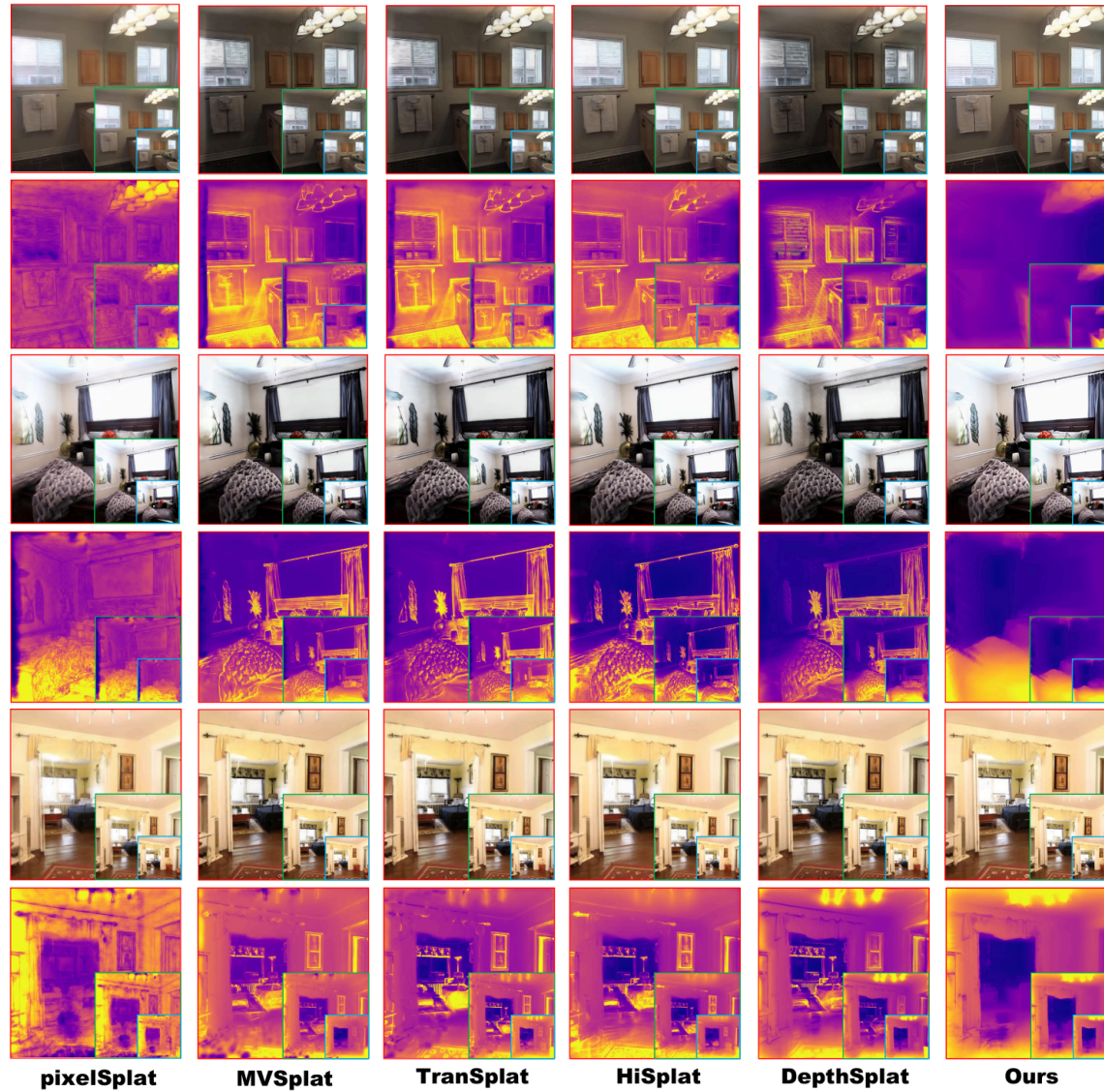
Experiments

Hyperparameter Summary

FAB(Forced Alpha Blending)

- Opacity bound: $\tau_{\text{opa}} = 0.6$
- Alpha threshold: Train 0.1 / Eval 0.001

Experiments



Experiments

Table 1: Novel view synthesis performance on the RealEstate10k dataset.

Method	256×256 (Standard)			512×512 (HRRC)			1024×1024 (HRRC)			Average		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
pixelSplat	26.049	0.862	0.137	25.782	0.868	0.207	24.920	0.877	0.269	25.584	0.869	0.204
HiSplat	27.193	0.882	0.117	25.269	0.870	0.198	24.262	0.878	0.248	25.575	0.877	0.188
MVSplat	26.359	0.868	0.129	20.408	0.809	0.290	17.966	0.755	0.425	21.578	0.811	0.281
TranSplat	26.687	0.875	0.125	20.610	0.815	0.286	18.154	0.761	0.427	21.817	0.817	0.279
DepthSplat	<u>27.504</u>	<u>0.890</u>	0.112	20.031	0.774	0.341	16.385	0.635	0.491	21.307	0.766	0.315
Ours-S	27.001	0.883	0.118	25.989	<u>0.860</u>	0.223	24.535	0.835	0.325	25.842	0.859	0.222
Ours-B	27.447	<u>0.890</u>	<u>0.113</u>	<u>26.280</u>	0.866	<u>0.218</u>	<u>24.744</u>	<u>0.838</u>	<u>0.322</u>	<u>26.157</u>	<u>0.865</u>	<u>0.217</u>
Ours-L	27.537	0.892	0.112	26.331	0.866	0.217	24.897	0.842	0.320	26.255	0.867	0.216

Table 2: Novel view synthesis performance on the ACID dataset.

Method	256×256 (Standard)			512×512 (HRRC)			1024×1024 (HRRC)			Average		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
pixelSplat	28.284	0.842	0.146	27.687	0.848	0.243	26.462	0.858	0.343	27.478	0.849	0.244
HiSplat	28.737	0.853	0.132	25.376	0.833	0.246	23.988	0.841	0.314	25.700	0.842	0.231
MVSplat	28.202	<u>0.842</u>	0.145	17.802	0.711	0.406	14.784	0.572	0.567	20.263	0.708	<u>0.373</u>
TranSplat	28.337	0.845	0.143	<u>17.911</u>	<u>0.716</u>	<u>0.402</u>	<u>14.956</u>	<u>0.582</u>	<u>0.558</u>	<u>20.401</u>	<u>0.714</u>	<u>0.373</u>
Ours	<u>28.336</u>	0.845	<u>0.144</u>	26.868	0.814	0.281	21.253	0.690	0.457	25.486	0.783	0.294

Experiments

Table 3: Cross datasets performance.

Method	Scannet			DL3DV			DTU			Average		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
pixelSplat	19.606	0.714	0.324	27.201	0.882	0.104	12.752	0.329	0.639	19.853	0.642	0.356
HiSplat	19.095	0.691	0.342	26.242	0.869	0.112	16.019	0.671	0.277	20.452	0.744	0.244
MVSplat	18.725	0.692	0.333	23.841	0.768	0.156	13.914	0.470	0.386	18.827	0.643	0.292
TranSplat	18.944	0.705	0.332	23.913	0.771	0.161	<u>14.956</u>	0.527	0.327	19.271	0.668	<u>0.273</u>
DepthSplat	<u>20.201</u>	0.735	0.305	28.141	0.905	0.083	14.592	0.425	0.436	<u>20.978</u>	<u>0.688</u>	0.275
Ours	20.305	<u>0.731</u>	<u>0.313</u>	<u>27.384</u>	<u>0.890</u>	<u>0.106</u>	15.544	<u>0.488</u>	<u>0.329</u>	21.078	0.703	0.249

Table 4: Ablations study on various components.

Method	256×256 (Standard)			512×512 (HRRC)			1024×1024 (HRRC)			Average		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
w/o FAB, SCP	26.925	0.880	0.120	21.549	0.805	0.307	18.563	0.716	0.422	22.346	0.800	0.283
w/o FAB	26.481	0.873	0.128	21.042	0.776	0.345	17.576	0.662	0.474	21.700	0.770	0.316
Full	27.001	0.883	0.118	25.989	0.860	0.223	24.535	0.835	0.325	25.842	0.859	0.222

Experiments

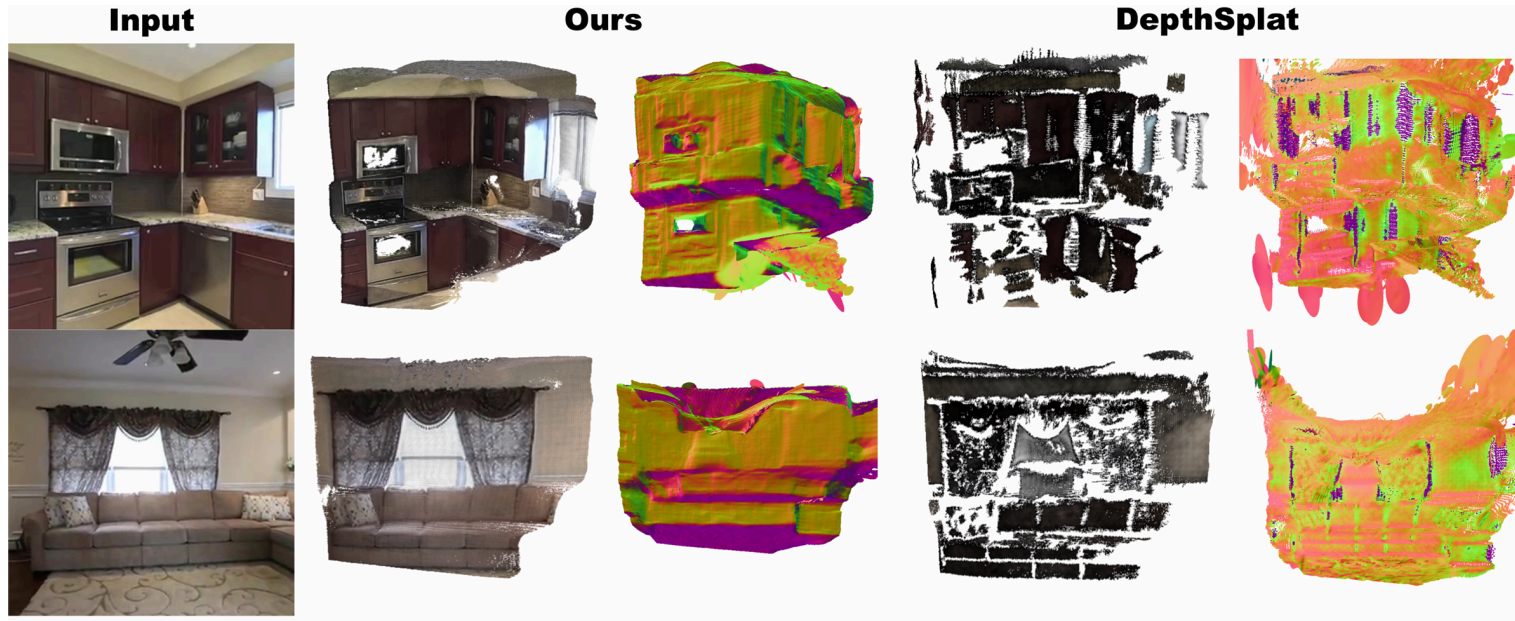


Figure 6: Normal and mesh comparison with DepthSplat.

Table 5: Quantitative performance comparison on high-resolution DL3DV dataset.

Metric	pixelSplat	HiSplat	MVSplat	TransSplat	DepthSplat	Ours
PSNR $\uparrow$	<u>24.082</u>	22.780	17.966	19.545	16.066	24.411
SSIM $\uparrow$	<u>0.755</u>	<u>0.765</u>	0.645	0.679	0.600	0.788
LPIPS $\downarrow$	0.250	<u>0.237</u>	0.301	0.257	0.424	0.252

Experiments

Table 6: Ablations study on hyperparameter sensitivity.

Method	256×256 (Standard)			512×512 (HRRC)			1024×1024 (HRRC)			Average		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
Ours	27.001	0.883	0.118	25.989	0.860	0.223	24.535	0.835	0.325	25.842	0.859	0.222
$\tau_\alpha = 0.3$	26.921	0.881	0.120	25.930	0.860	0.222	24.816	0.843	0.317	25.889	0.861	0.220
$\tau_{\text{opa}} = 0.4$	26.992	0.883	0.118	25.957	0.860	0.222	24.538	0.835	0.323	25.829	0.859	0.221

Conclusion

- Feedforward 방식의 최신 논문
 - 이 분야의 논문은 한번에 잘 만드는 것이 관건
- Surface continuity prior를 도입하여, Gaussian의 rotation, scale을 더 정확하게 추정
- Forced alpha blending을 통해, occlusion 문제 해결
- HRRC metric을 도입하여, 고해상도에서의 rendering 품질 평가
- SLAM에서 비교해보면, frontend라고 생각할 수 있음
 - 최적화는 생각하지 않고, scene을 확장해나가고 odometry 계산에 초점
 - 다만 품질은 떨어질 수 있음
- Transformer 기반의 feedforward, frontend 방식이 괜찮은가?
 - Optimization의 계산비용과 품질 사이의 trade-off
 - Kalman filter?